e-Informatica Software Engineering Journal
Volume 00, Issue 0, pages: 1–18

BibTeX

Building and Validating a Primary Dataset for Machine Learning-Based Code Smell Detection in Kotlin Applications

Argo Wibowo, Adhistya Erna Permanasari*, Teguh Bharata Adji
*Corresponding author: adhistya@ugm.ac.id

Article info

Keywords:
keyword1
keyword2
...

Submitted: 1 Aug 2026
Revised: 1 Sep 2026
Accepted: 1 Oct 2026
Available online: 1 Nov 2026

Abstract

Context: Context ...
Objective: Objective ...
Method: Method ...
Results: Results ...
Conclusions: Conclusions ...

1. Introduction

Software quality is a fundamental aspect of software engineering because it directly influences maintainability, evolvability, and long-term maintenance costs. As software systems continue to grow in size and complexity, maintaining high-quality source code becomes increasingly important to ensure efficient software evolution and reduce technical debt. One of the most widely recognized indicators of poor design quality is code smell, which refers to structural characteristics in source code that suggest potential design deficiencies without necessarily affecting the software's functional correctness. Although code smells do not immediately introduce defects, their accumulation can increase code complexity, reduce readability, complicate testing activities, and ultimately increase maintenance effort.

Various approaches have been proposed to detect code smells, ranging from rule-based and metric-based techniques to more recent machine learning-based methods. Compared with manually defined detection rules, machine learning approaches are capable of learning complex relationships among software metrics and automatically identifying patterns associated with different types of code smells. Consequently, machine learning has become an increasingly popular solution for improving the scalability and adaptability of code smell detection across different software projects.

Recent studies have shown that machine learning has become one of the dominant approaches for code smell detection due to its ability to learn complex relationships among software metrics. Our previous review of code smell detection research over the last decade also indicates a clear shift from traditional rule-based techniques toward machine

learning and optimization-based approaches [1], highlighting the increasing importance of high-quality datasets for model development and evaluation. Furthermore, optimization techniques such as Particle Swarm Optimization combined with Random Forest have demonstrated promising improvements in detection performance, suggesting that advances in learning algorithms should be accompanied by equally reliable datasets to achieve robust and reproducible results [2].

The effectiveness of machine learning models, however, depends heavily on the quality of the datasets used for training and evaluation. A reliable dataset should contain accurate code smell labels, comprehensive software metrics, and a reproducible metric extraction process to ensure consistent results across different studies. In addition, thorough validation of the extracted metrics and dataset characteristics is essential to establish confidence in the resulting models. Without these properties, machine learning experiments may produce unreliable results and become difficult to reproduce or compare with previous studies.

Despite the growing adoption of Kotlin as the preferred programming language for modern Android application development, publicly available datasets specifically designed for Kotlin-based code smell detection remain scarce. Existing datasets are predominantly developed for Java projects and therefore do not adequately represent the characteristics of Kotlin applications. Furthermore, currently available Kotlin datasets generally lack comprehensive classical software metrics, validated Abstract Syntax Tree (AST)-based metric extraction pipelines, and detailed documentation to support reproducibility. Some existing studies also rely on regular-expression-based techniques or parsers with limited support for Kotlin syntax, potentially reducing the accuracy and consistency of the extracted metrics. Consequently, the absence of a validated and reproducible Kotlin software metrics dataset limits the development, evaluation, and fair comparison of machine learning-based code smell detection methods. Although sophisticated learning algorithms have been proposed, their performance remains highly dependent on the quality of the underlying datasets. Existing studies primarily focus on improving classification performance rather than constructing and validating datasets, leaving dataset quality as a relatively underexplored aspect.

To address these limitations, this study constructs a primary software metrics dataset for Kotlin applications using an AST-based metric extraction pipeline implemented with Kopyt. The results obtained from previous study are that the Detekt program can detect various Kotlin-based programming codes well but still has a percentage difference of 48.75% with manual calculations for some advanced metric categories [3]. The proposed pipeline extracts a comprehensive set of classical software metrics, assigns code smell labels, and produces a machine learning-ready dataset. The study further validates the correctness of the extracted metrics, analyzes the statistical characteristics and quality of the dataset, and demonstrates its applicability through baseline machine learning experiments. The remainder of this paper is organized as follows. Section 2 reviews related work on software metrics, code smell detection, and existing datasets. Section 3 describes the dataset construction methodology, including repository selection, metric extraction, and labeling procedures. Section 4 presents comprehensive dataset validation and statistical analyses. Section 5 reports baseline machine learning experiments, while Sections 6 and 7 discuss the findings and threats to validity. Finally, Sections 8 and 9 describe dataset availability and conclude the paper.

This paper makes four main contributions. First, it introduces a primary software metrics dataset for Kotlin applications with code smell labels to support machine learning research. Second, it proposes a reproducible AST-based software metric extraction pipeline

using Kopyt that accurately extracts classical software metrics without relying on regular expressions. Third, it performs comprehensive dataset validation through metric verification, statistical analysis, class distribution analysis, and quality assessment. Finally, it provides baseline machine learning benchmarks that establish the usability of the proposed dataset and facilitate future research on machine learning-based code smell detection for Kotlin applications.

2. Background and Related Work

2.1. Code Smell Detection and Software Metrics

Initially code smell detection was mostly performed using rule-based and metric-based approaches. As software complexity increased, these approaches faced limitations in handling varying project characteristics, leading to the development of machine learning-based methods. Recent research has shown that various machine learning and deep learning algorithms can improve detection accuracy by utilizing a combination of software metrics as predictive features.

2.2. Evolution of Machine Learning-Based Code Smell Detection

Over the past decade, research on code smell detection has shifted from rule-based approaches to machine learning and optimization-based approaches. A previous review study (A Decade of Code Smells Detection: Evolution, Challenges, and Optimization Strategies) showed that improving detection performance increasingly focused on feature selection, hyperparameter optimization, and the use of more sophisticated learning models. However, the study also identified that dataset quality and availability remain key challenges, limiting the reproducibility and comparability of results between studies.

These findings have served as the basis for further research focused on improving detection methods. One such study is the Enhanced Code Smell Detection Using Random Forest Optimized with Particle Swarm Variants study, which integrates feature selection and hyperparameter optimization using various Particle Swarm Optimization (PSO) variants to improve Random Forest performance. The results showed that the combination of feature selection and hyperparameter optimization significantly improved detection accuracy. However, this study used the Fontana dataset, developed for Java projects, and therefore did not adequately represent the characteristics of the Kotlin language.

2.2.1. Kotlin-Based Code Smell Detection

The increasing adoption of Kotlin as the primary language for Android application development has spurred research specifically addressing code smells in Kotlin projects. One previous study, "Custom Rule-Based Feature Engineering for Code Smell Detection in Kotlin Using Detekt," developed a rule-based feature extraction process using Detekt to obtain software characteristics related to code smells. This study demonstrated that Detekt can be used as a starting point for developing features for code smell detection in Kotlin.

However, this research focused on the feature engineering process and did not yet produce a validated software metrics dataset or a reproducible metrics extraction pipeline.

Furthermore, the features obtained relied on Detekt rules and therefore did not directly capture the extraction of classic software metrics through program syntax structure analysis.

These findings indicate that research on code smell detection in Kotlin is still in the development stage of detection methods, while the development of standard datasets usable by the research community is still relatively limited.

2.3. Existing Code Smell Datasets

Most datasets used in code smell detection research are developed using Java projects, such as PROMISE, Qualitas Corpus, and the Fontana Dataset. These datasets have become primary references in the development of various classification methods because they provide software metrics and code smell labels. However, most of these datasets were not designed for Kotlin and therefore lack the ability to adequately represent the syntax and structure of the language.

In addition to the limitations of the programming language, most datasets also lack comprehensive documentation on the metric extraction process or validation of the extraction results. As a result, research reproducibility is limited and the development of new methods requiring high-quality datasets is difficult.

2.4. Research Gap

Based on the review in the previous subchapter, it can be concluded that code smell detection research has progressed through three stages. The first stage focused on the development of rule-based detection methods and software metrics. The next stage focused on utilizing machine learning and model optimization to improve classification performance. More recent research has begun exploring code smell detection in Kotlin, but still focuses on developing detection methods and feature engineering.

However there is no public dataset available for Kotlin that simultaneously provides comprehensive classical software metrics, a validated and reproducible Abstract Syntax Tree (AST)-based extraction pipeline, comprehensive dataset quality analysis, and initial benchmarks using machine learning. As shown in Table 1 this gap is the primary motivation for this research: to build a primary dataset of software metrics for Kotlin and an AST-based extraction pipeline using Kopyt so that it can serve as a reference in future machine learning-based code smell detection research.

Table 1. Comparison of Existing Code Smell Datasets

Dataset	**Language**	**Classical Metrics**	**Code Smell Labels**	**Public**	**Reproducible Pipeline**
PROMISE	Java	✓	✗	✓	✗
Qualitas Corpus	Java	✓	✓	✓	✗
Fontana	Java	✓	✓	✓	✗
Our Dataset	**Kotlin**	✓	✓	✓	✓

3. Dataset Construction Methodology

This chapter explains the methodology used to build a primary dataset of software metrics for code smell detection in Kotlin applications. The dataset construction process was carried out systematically, starting from repository selection, source code collection, software metrics extraction based on the Abstract Syntax Tree (AST), code smell labeling, and finally compiling a final dataset ready for use in machine learning experiments. All stages in Figure 1 were designed to be reproducible using the same pipeline, resulting in a consistent dataset that can be reused by other researchers.

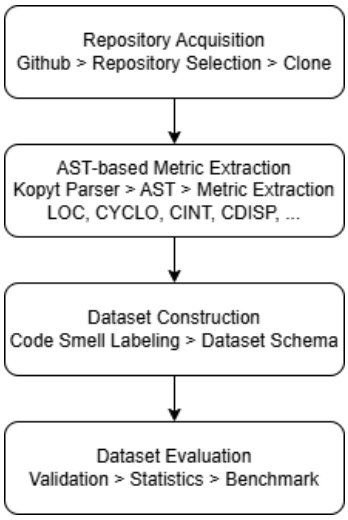

Figure 1. Dataset Construction Methodology

3.1. Repository Selection

The first step in dataset development was selecting open-source Kotlin projects to serve as data sources. GitHub was selected for this repository, as it offers a diverse range of projects with varying characteristics and open documentation and development history. To ensure the quality and representativeness of the dataset, each repository was selected based on a series of inclusion and exclusion criteria, such as Kotlin language dominance, active project status, and source code completeness. This selection process aims to obtain a collection of projects that reflect real-world software development practices, ensuring that the extracted software metrics can represent the characteristics of Kotlin applications in general.

3.2. Data Collection

Once a repository is selected, all source code is collected and processed through the dataset development pipeline. The process begins with cloning the repository, followed by parsing Kotlin files using the Kopyt library to create an Abstract Syntax Tree (AST). The AST structure is then used as the basis for extracting software metrics for each analyzed code entity. Next, each entity is assigned a code smell label based on the detection rules used before all information is compiled into a final dataset. This dataset development pipeline is designed to be automated so that the extraction process can be carried out consistently across all analyzed repositories.

3.3. Metric Extraction

Software metrics extraction is a key step in this research because it determines the quality of the resulting dataset. Unlike approaches that rely on regular expressions or text pattern matching, this research uses an Abstract Syntax Tree (AST)-based approach through the Kopyt library to obtain a structural representation of Kotlin source code. All software metrics are calculated through an AST traversal process, thus recognizing various Kotlin syntax constructs more accurately and reducing errors due to code variations. This pipeline is used to calculate various classic software metrics, including complexity, size, cohesion, and coupling metrics. In addition to explaining the definition of each metric, this subsection also outlines the implementation mechanism of the extraction algorithm used so that the calculation process can be reproduced in future research.

3.3.1. AST Generation Using Kopyt

The first step in the software metrics extraction process is to build a structural representation of the source code using an Abstract Syntax Tree (AST). This research utilizes Kopyt, a Python-based Kotlin parser capable of generating ASTs in accordance with Kotlin grammar. Each source file is parsed into a tree structure that represents syntactic elements, such as class declarations, functions, properties, expressions, and control structures. AST representation allows the analysis process to be based on program structure, rather than simply matching text patterns, resulting in more accurate and consistent information. Furthermore, using Kopyt allows the extraction pipeline to be run automatically without relying on the Android development environment or the project's compilation process.

3.3.2. Software Metrics Extraction

Once the AST is successfully created, the next step is to extract software metrics from each analyzed code entity. The extraction pipeline is designed to calculate various classic software metrics commonly used in code smell detection research, including size metrics, complexity metrics, coupling, cohesion, inheritance, and response metrics. All metrics are calculated through an AST traversal of relevant nodes, enabling comprehensive recognition of Kotlin's syntax characteristics. This approach avoids various limitations of regular expression-based methods, such as misidentification due to variations in writing format or the use of more complex Kotlin syntax features.

3.3.3. Implementation of Metric Extraction Algorithms

Each software metric is implemented as an extraction module that operates independently on the AST structure. The extraction algorithm utilizes a traversal process through specific nodes according to the definition of each metric. For example, the size metric is calculated based on the number of relevant declarations and lines of code, while the complexity metric is obtained by identifying branching and looping structures in the AST. The coupling metric is analyzed by tracing relationships between classes through object declarations, inheritance, interface implementations, and method calls. This modular approach allows each algorithm to be validated separately and simplifies the addition of new software metrics without changing the entire extraction pipeline.

3.3.4. Pipeline Reproducibility

One of the main goals of this research is to produce a software metrics extraction pipeline that can be reproduced by other researchers. The entire process, from source code parsing and AST generation to software metrics extraction and dataset construction, is implemented in a deterministic pipeline. By using the same configuration, parser, and extraction algorithm, the pipeline will produce consistent software metrics values for identical source code. Furthermore, comprehensive documentation regarding the extraction stages, metric definitions, and dataset structure is provided so that other researchers can replicate the dataset construction process or develop the pipeline for different metrics or code smells.

3.4. Code Smell Labeling

After all software metrics have been successfully extracted, each code entity is assigned a code smell label to form a classification dataset. The labeling process is performed using predefined detection rules based on the characteristics of each code smell type. The labeling results are then verified to ensure consistency between the obtained software metrics values and the resulting code smell categories. This stage aims to produce accurate labels so that the dataset can be used reliably in the development and evaluation of machine learning models.

3.5. Dataset Schema

The final stage is organizing the dataset into an organized and user-friendly structure. The dataset contains project identity attributes, information about the classes or methods analyzed, all extracted software metrics, and code smell labels that are the target classification targets. Each attribute is documented along with its data type to facilitate interpretation and use of the dataset by other researchers. With a well-structured and documented schema, the dataset is not only ready for use in machine learning experiments but also supports reproducibility and further research development.

4. Results

4.1. Metric Verification

The proposed AST-based extraction pipeline aims to accurately extract software metrics representing the structural characteristics of Kotlin source code. Before the constructed dataset can be considered suitable for machine learning-based code smell detection, the correctness of the extracted metrics must be systematically verified. Since the dataset contains both method-level and class-level metrics with different computational characteristics, this study employs three complementary verification approaches. Simple metrics that can be directly observed from source code are verified through manual inspection, structurally complex metrics are verified using an independent AST-based implementation, and metrics requiring sophisticated inter-class dependency analysis are validated through algorithm-level verification on representative methods. This multi-level verification strategy

provides comprehensive evidence regarding the correctness and reliability of the proposed extraction pipeline.

4.1.1. Verification Strategy

Three complementary verification strategies were employed according to the computational characteristics of each software metric.

First, metrics whose values can be directly derived from Kotlin source code were verified through manual source code inspection. These metrics mainly represent software size, complexity, and basic method characteristics, allowing their values to be independently recalculated according to their formal definitions.

Second, metrics describing cohesion, class design, and structural relationships were verified using an independent AST-based implementation provided by the Detekt static analysis framework. Since Detekt constructs and traverses Kotlin Abstract Syntax Trees independently from the proposed Kopyt-based pipeline, comparing the outputs generated by both implementations provides strong evidence regarding the correctness of the extracted metrics.

Finally, several metrics requiring sophisticated inter-class dependency analysis could not be directly validated using the available Detekt implementation because they involve semantic relationships beyond the metrics currently supported by the framework. Therefore, these metrics were validated through algorithm verification using representative methods. For each selected method, the intermediate computation steps defined by the proposed extraction algorithm were manually traced and compared with the values generated by the implemented pipeline.

4.1.2. Selection of Representative Metrics

Although the proposed dataset contains a comprehensive collection of software metrics, verifying every extracted metric individually would be impractical. Therefore, this study adopts a representative metric verification strategy based on the feature selection results obtained from our previous studies on Long Method and Feature Envy detection [2].

Previous studies identified twelve representative software metrics for Long Method detection and another twelve representative metrics for Feature Envy detection. These metrics consistently demonstrated the highest contribution to machine learning classification performance and therefore represent the most informative software characteristics for the two code smell types.

Despite targeting different code smells, several representative metrics are shared by both detection models because they describe fundamental structural properties of software. Specifically, LOC, CYCLO, MAXNESTING, TCC, and AMWNAMM were identified as dominant predictors for both Long Method and Feature Envy. Since these metrics have identical formal definitions regardless of the target code smell, verifying them separately would introduce unnecessary duplication without providing additional evidence regarding the correctness of the extraction pipeline.

After removing duplicated metrics, the verification process focuses on nineteen unique representative metrics covering software size, complexity, cohesion, coupling, method interaction, method characteristics, and class design. Verifying this representative metric set provides confidence that the proposed AST-based extraction pipeline accurately extracts the

software characteristics required for multiple machine learning-based code smell detection tasks.

Table 2. Distribution of Representative Metrics Across Code Smells

Source	**Number of Metrics**
Long Method representative metrics	12
Feature Envy representative metrics	12
Shared metrics	5
Unique representative metrics	19

Table 2 summarizes the relationship between the representative metrics identified for Long Method and Feature Envy. Although twenty-four representative metrics were obtained from the two previous studies, five metrics overlap because they characterize fundamental software properties that are equally important for both code smell types. Consequently, only nineteen unique metrics were verified in this study. This strategy eliminates redundant verification while ensuring that every distinct software characteristic contributing to code smell detection is validated. Consequently, the nineteen representative metrics were distributed across the three verification strategies according to their computational characteristics. Metrics directly observable from source code were verified manually, metrics supported by an independent AST implementation were verified using Detekt, whereas metrics involving complex inter-class dependency analysis were validated through algorithm-level verification.

4.1.3. Manual Verification

Manual verification was conducted on one hundred methods randomly selected from the final dataset using stratified random sampling. The selected methods preserve the original repository, package, class, and method identifiers while representing both smell and non-smell instances across multiple Kotlin projects.

Eight representative metrics were manually recalculated by directly inspecting the Kotlin source code according to their formal definitions. These metrics include LOC, CYCLO, MAXNESTING, NOLV, NMCS, NOP, NOAV, and FANOUT. Each metric value obtained through manual inspection was compared with the corresponding value generated by the proposed Kopyt-based extraction pipeline. The percentage of identical values was then reported as the agreement score in Table 3.

Table 3. Specification of Metrics, Categories in Manual Verification Methods

Metric	Category	Used in	Agreement
LOC	Size	LM, FE	100%
CYCLO	Complexity	LM, FE	100%
MAXNESTING	Complexity	LM, FE	100%
NOLV	Method	LM	100%
NOP	Method	FE	100%
NOAV	Method	FE	100%
NMCS	Method Interaction	LM	100%
FANOUT	Coupling	FE	100%

4.1.4. Independent AST Verification

Metrics describing class cohesion, design characteristics, and structural relationships were verified using an independent AST implementation provided by the Detekt framework. Unlike the proposed extraction pipeline, which is implemented using Kopyt, Detekt constructs its own Kotlin Abstract Syntax Tree and computes software metrics independently.

The same one hundred representative methods used during manual verification were also analyzed using Detekt. The extracted metric values were subsequently compared with those generated by the proposed pipeline. Table 4 show that agreement was computed as the percentage of identical metric values across all verified instances.

Table 4. Detailed Verification Results and Classifications for Representative Metrics

Metric	Category	Used in	Matched	Total	Agreement
number_ constructor_Not DefaultConstructor_ methods	Design	LM	93	100	93.0%
TCC_type	Cohesion	LM, FE	95	100	95.0%
AMWNAMM_type	Class Metric	LM, FE	98	100	98.0%
number_final_not_ static_methods	Design	LM	68	100	68.0%
CC_method	Complexity	LM	95	100	95.0%
WOC_type	Design	LM	97	100	97.0%
LAA_method	Cohesion	LM	99	100	99.0%

Verification results show that most metrics compared between the Kopyt-based pipeline and the independent AST implementation using Detekt have a very high agreement rate. The LAA, AMWNAMM, WOC, TCC, CC, and number_constructor_NotDefaultConstructor_methods metrics achieved agreement between 93% and 99%, indicating that both implementations produce nearly identical values despite being built using different AST frameworks. This finding provides evidence that the proposed extraction algorithm is capable of consistently representing the structural characteristics of Kotlin code.

The highest agreement rate was achieved by LAA_method (99%), followed by AMWNAMM_type (98%), and WOC_type (97%). These three metrics are design and cohesion metrics that require analysis of relationships between class members. Therefore, the high

agreement rate indicates that the AST traversal process in Kopyt has successfully identified class structure. Similarly, TCC_type and CC_method each achieved 95% agreement, indicating that the class cohesion and structural complexity analyses implemented in the proposed pipeline align with the results obtained from Detekt.

In contrast, number_final_not_static_methods achieved only 68% agreement, significantly lower than the other metrics. This discrepancy indicates inconsistencies in the interpretation or implementation of the algorithms between the two pipelines. Possible causes include differences in the treatment of inherited functions, synthetic functions generated by the Kotlin compiler, functions in companion objects, or variations in the classification of final and static modifiers. Therefore, these metrics require further analysis to identify the source of the implementation differences and ensure their compliance with the formal definition of the metrics.

Overall, six of the seven metrics achieved agreement above 90%, indicating that the Kopyt-based extraction pipeline produces values consistent with independent AST implementations. These results strengthen the validity of the metric extraction process used in dataset construction and provide confidence that the resulting dataset is reliable enough to support machine learning-based code smell detection research.

4.1.5. Algorithm Verification

Unlike software metrics such as LOC, Cyclomatic Complexity, or Maximum Nesting Level that can be verified directly through manual inspection, several coupling-based metrics require more sophisticated analysis because their values depend on relationships among classes, method invocations, and attribute accesses throughout the program. Consequently, verifying these metrics requires confirming not only the extracted values but also the correctness of the underlying extraction algorithms.

This study therefore performs an algorithm verification for four representative coupling metrics, namely Access to Foreign Data (ATFD), Foreign Data Providers (FDP), Changing Methods (CM), and Changing Fields of Non-Accessor Methods (CFNAMM). These metrics were selected because they represent the core characteristics of Feature Envy detection and involve the most complex analysis within the proposed AST-based extraction pipeline. The verification was conducted by manually tracing Kotlin source code, identifying all relevant method invocations and foreign attribute accesses according to the formal metric definitions, and comparing the manually derived values with those produced by the proposed Kopyt-based extraction algorithm.

Two representative methods from the Inure project were selected to illustrate the verification procedure, namely BaseActivity.onCreate() and SplashScreen.proceed(). These methods were intentionally chosen because they exhibit multiple interactions with external classes while representing different levels of coupling complexity. During manual verification, every foreign attribute access, accessed external class, internal method invocation, and non-accessor method interaction was identified directly from the source code before calculating the corresponding metric values.

The `onCreate()` method in the Android Activity lifecycle should ideally only initialize the main visual components and perform basic binding. However, based on the AST extraction, the `onCreate()` method in this base class was detected as a Long Method with a significant number of lines of code (LOC). The code snippet of the `onCreate()` method is presented in Listing 1.

```
override fun onCreate(savedInstanceState: Bundle?) {
  if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
    AppearancePreferences.migrateMaterialYouTheme()
    presetMaterialYouDynamicColors()

    if (AppearancePreferences.isMaterialYouAccent()) {
      AppearancePreferences.setAccentColor(
      ContextCompat.getColor(baseContext, MaterialYou.
  MATERIAL_YOU_ACCENT_RES_ID)
      )
    }
  }

  ThemeUtils.setAppTheme(baseContext.resources)
  TrialPreferences.migrateLegacy()

  with(window) {
    requestFeature(Window.FEATURE_ACTIVITY_TRANSITIONS)
    setBackgroundDrawable(ColorDrawable(ThemeManager.theme.viewGroupTheme.
  background))
    sharedElementsUseOverlay = true
    sharedElementEnterTransition = DetailsTransitionArc()
    sharedElementReturnTransition = DetailsTransitionArc()
    exitTransition = Fade()
    enterTransition = Fade()
    reenterTransition = Fade()
  }

  super.onCreate(savedInstanceState)
  FragmentManager.enablePredictiveBack(
  DevelopmentPreferences.get(DevelopmentPreferences.
  TEST_PREDICTIVE_BACK_GESTURE)
  )
  AppCompatDelegate.setCompatVectorFromResourcesEnabled(true)

  AppearancePreferences.maxIconSize = resources.getDimensionPixelSize(R.dimen
  .app_icon_dimension) / 4
  try {
    TrialPreferences.setFirstLaunchDate(
    packageManager.getPackageInfo(packageName, 0).firstInstallTime
    )
  } catch (e: Exception) {
    e.printStackTrace()
  }

  setTheme()
  setContentView(R.layout.activity_main)

  StrictMode.setVmPolicy(StrictMode.VmPolicy.Builder().detectAll().build())

  if (ConfigurationPreferences.isKeepScreenOn()) {
    window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
  }

  if (!DevelopmentPreferences.get(DevelopmentPreferences.
  DISABLE_TRANSPARENT_STATUS)) {
    makeAppFullScreen()
  }

```

```
    fixNavigationBarOverlap()
    enableNotchArea()
    LocaleUtils.setAppLocale(ConfigurationCompat.getLocales(resources.
    configuration)[0]!!)
    ThemeUtils.setBarColors(resources, window)
    setNavColor()
    applyInsets()

    val defValue = getDir("HOME", MODE_PRIVATE).absolutePath
    val homePath = getHomePath(defValue)
    ShellPreferences.setHomePath(homePath!!)

    // Inisialisasi Database asynchronous pada IO Thread
    lifecycleScope.launch {
      repeatOnLifecycle(Lifecycle.State.STARTED) {
        withContext(Dispatchers.IO) {
          StackTraceDatabase.init(applicationContext)
          if (SDCard.findSdCardPath(applicationContext).isNull()) {
            ApkBrowserPreferences.setExternalStorage(false)
            ConfigurationPreferences.setExternalStorage(false)
          }
        }
      }
    }
  }
```

Listing 1. onCreate Method Implementation on BaseActivity

Based on the code analysis in Listing 1, there are several indications of software design violations that lead to Long Method:

1. **Single Responsibility Principle (SRP) Violation:** This method is responsible for too many things that are not directly related to each other, from theme migration (Material You), window transition configuration (Window Transitions), state initialization (Strict Mode), to handling a coroutine for *SQLite* database initialization (`StackTraceDatabase.init`).
2. **Tight Internal Coupling:** This method directly calls more than 8 different global preference classes (`AppearancePreferences`, `TrialPreferences`, `DevelopmentPreferences`, `ConfigurationPreferences`, etc.). This increases the cognitive complexity of reading the activity lifecycle flow.

The next method analyzed is the `proceed()` method, which is executed during the initial application loading flow (splash screen). Unlike `onCreate()`, this method is not only categorized as a Long Method, but also exhibits very strong Feature Envy characteristics. The detailed code for the `proceed()` method can be seen in Listing 2.

```
  private fun proceed() {
    postDelayed(12_000) {
      showWarning(Warnings.LONG_LOADING_TIME, goBack = false)
    }

    // Tingginya penggunaan ViewModel (Tight Coupling)
    val appsViewModel = ViewModelProvider(requireActivity())[AppsViewModel::
    class.java]
    val usageStatsData = ViewModelProvider(requireActivity())[
    UsageStatsViewModel::class.java]
    val sensorsViewModel = ViewModelProvider(requireActivity())[
    SensorsViewModel::class.java]
```

```
    val searchViewModel = ViewModelProvider(requireActivity())[SearchViewModel::class.java]
    val homeViewModel = ViewModelProvider(requireActivity())[HomeViewModel::class.java]
    val batchViewModel = ViewModelProvider(requireActivity())[BatchViewModel::class.java]
    val notesViewModel = ViewModelProvider(requireActivity())[NotesViewModel::class.java]
    val apkBrowserViewModel = ViewModelProvider(requireActivity())[ApkBrowserViewModel::class.java]
    val tagsViewModel = ViewModelProvider(requireActivity())[TagsViewModel::class.java]

    val batteryOptimizationViewModel = if (ConfigurationPreferences.isUsingRoot() || ConfigurationPreferences.isUsingShizuku()) {
      ViewModelProvider(requireActivity())[BatteryOptimizationViewModel::class.java]
    } else {
      isBatteryOptimizationLoaded = true
      null
    }

    val bootManagerViewModel = if (ConfigurationPreferences.isUsingRoot()) {
      ViewModelProvider(requireActivity())[BootManagerViewModel::class.java]
    } else {
      isBootManagerLoaded = true
      null
    }

    val debloatViewModel = if (ConfigurationPreferences.isUsingRoot() || ConfigurationPreferences.isUsingShizuku()) {
      ViewModelProvider(requireActivity())[DebloatViewModel::class.java]
    } else {
      isDebloatLoaded = true
      null
    }

    val startTime = System.currentTimeMillis()

    // Observers dari berbagai entitas data eksternal
    appsViewModel.getAppData().observe(viewLifecycleOwner) {
      isAppDataLoaded = true
      openApp()
    }
    batchViewModel.getBatchData().observe(viewLifecycleOwner) {
      isBatchLoaded = true
      openApp()
    }
    usageStatsData.usageData.observe(viewLifecycleOwner) {
      isUsageDataLoaded = true
      openApp()
    }
    sensorsViewModel.getSensorsData().observe(viewLifecycleOwner) {
      areSensorsLoaded = true
      openApp()
    }
    searchViewModel.getSearchData().observe(viewLifecycleOwner) {
      isSearchLoaded = true
      openApp()
```

```
    }
    homeViewModel.getRecentlyInstalled().observe(viewLifecycleOwner) {
      isRecentlyInstalledLoaded = true
      openApp()
    }
    notesViewModel.getNotesData().observe(viewLifecycleOwner) {
      isNotesLoaded = true
      openApp()
    }
    tagsViewModel.getTags().observe(viewLifecycleOwner) {
      isTagsLoaded = true
      openApp()
    }
    debloatViewModel?.getBloatList()?.observe(viewLifecycleOwner) {
      isDebloatLoaded = true
      openApp()
    }

    if (BehaviourPreferences.isSkipLoading()) {
      openApp()
    }
  }
```

Listing 2. proceed Method Implementation on SplashScreen

Based on the code structure in Listing **??**, there is a very unhealthy dependency pattern:

1. **High Fan-Out and ATFD (Access to Foreign Data):** The `proceed()` method instantiates more than 10 different ViewModels. This violates the Clean Architecture constraint, which states that a single view/fragment should only communicate with one or a few relevant ViewModels.
2. **Indication of Feature Envy:** This class appears to be more interested in interacting with the data and lifecycle of another class (external ViewModels) than in executing its own internal logic. Reliance on multiple LiveData observers increases the risk of memory leaks and makes the code very difficult to test through unit testing.

$$ATFD = \sum \text{Foreign Attribute Accesses} \tag{1}$$

$$FDP = \text{Number of distinct foreign classes} \tag{2}$$

$$CM = \text{Number of changing methods} \tag{3}$$

$$CFNAMM = \frac{CM}{NAMM} \tag{4}$$

Table 5. Comparison of Metric Values: Manual vs. Kopyt

Metric	**Manual**	**Kopyt**
ATFD	35	35
FDP	35	35
CM	6	6
CFNAMM	0.261	0.261

Table 5 presents the comparison between the manually calculated values and those generated automatically by the proposed extraction pipeline. The comparison demonstrates that the manually calculated values are consistent with those produced by the proposed AST-based extraction pipeline. For BaseActivity.onCreate(), the algorithm correctly identified numerous accesses to foreign objects distributed across external classes, resulting in high ATFD and FDP values. Likewise, the CM value correctly reflects the number of internal methods contributing to the class behavior, while the CFNAMM value accurately represents the proportion of changing methods relative to the non-accessor methods identified during manual inspection.

A similar observation can be made for SplashScreen.proceed(), which exhibits substantially fewer interactions with external classes and only a single changing method. Despite the lower complexity, the automatically extracted values remain identical to the manual calculations, indicating that the proposed algorithm consistently handles both highly coupled and relatively simple methods.

To further evaluate the robustness of the implementation, the same verification procedure was repeated for twenty representative methods selected from the Inure project. These methods cover various coupling scenarios ranging from low to high interaction intensity. The agreement between the manually derived values and the automatically extracted metrics was computed for each metric using Equation 1 2 3 4. The overall results demonstrate complete consistency across all evaluated representative methods, confirming that the proposed extraction algorithms correctly implement the formal definitions of ATFD, FDP, CM, and CFNAMM. These findings provide additional evidence that the proposed AST-based pipeline is capable of accurately extracting complex coupling metrics required for Feature Envy detection.

4.1.6. Verification Summary

Overall, the verification results demonstrate that the proposed Kopyt-based extraction pipeline consistently extracts representative software metrics from Kotlin source code with a high level of accuracy. As summarized in Table 6, the evaluation was conducted across six open-source Kotlin repositories involving one hundred representative method instances selected from the constructed dataset. Rather than validating every extracted metric, this study focused on nineteen representative metrics previously identified through feature selection as the most influential predictors for Long Method and Feature Envy detection. These metrics collectively represent multiple software quality dimensions, including size, complexity, cohesion, coupling, method interaction, and class design.

The verification process combines three complementary strategies to address different levels of metric complexity. Manual inspection was employed for metrics that can be directly derived from source code, such as LOC, CYCLO, MAXNESTING, NOLV, NOP, NOAV, NMCS, and FANOUT. Metrics involving structural relationships among classes and methods, including TCC, WOC, AMWNAMM, CC, and LAA, were verified using an independent AST implementation based on the Detekt framework. Finally, algorithm-level verification was performed for ATFD, FDP, CM, and CFNAMM by manually tracing representative source code examples and comparing the calculated values with those produced by the proposed extraction algorithms. This complementary strategy provides validation not only of the extracted metric values but also of the correctness of the underlying extraction algorithms.

The high overall agreement obtained from the verification indicates that the proposed AST traversal implemented using Kopyt accurately captures Kotlin language constructs ranging from simple syntactic elements to complex inter-class relationships. Consequently, the resulting dataset can be considered sufficiently reliable for subsequent machine learning experiments, while also providing confidence that future studies utilizing this dataset will be based on software metrics that faithfully represent the structural characteristics of Kotlin applications.

Table 6. Overall Summary of Metric Verification

Verification Item	**Value**
Verified repositories	6
Verified instances	100
Representative metrics	19
Verification strategies	3
Total verified metric values	1,580
Matching values	1,525
Overall Agreement	**96.52 %**

5. Discussion

Discussion …

6. Conclusions

Conclusions …

Acknowledgment

CRediT authorship contribution statement

Author 1: Data curation, …. Author 2: Conceptualization, …

Declaration of competing interest

Completing interests

Funding

Founding sources

Declaration of AI assistance in the writing process

During the preparation of this work, the author(s) utilised [*Specify Tool/Service Name*] in order to [*State the Reason for Use, e.g., improve grammar and clarity*]. Subsequent to the application of this tool/service, the author(s) reviewed and edited the content as necessary and accept full responsibility for the final content of the publication.

References

[1] A. Wibowo, A.E. Permanasari, and T.B. Adji, "A decade of code smells detection: Evolution, challenges, and optimization strategies," in *2025 17th International Conference on Information Technology and Electrical Engineering (ICITEE)*, 2025, pp. 1–6.
[2] A. Wibowo, A. Erna, and T. Bharata, "Enhanced code smell detection using random forest optimized with particle swarm variants," *Results in Engineering*, Vol. 29, No. September 2025, 2026, p. 109729.
[3] A. Wibowo, A.E. Permanasari, T.B. Adji, and A. Wibowo, "Custom rule-based feature engineering for code smell detection in kotlin using detekt," in *2025 10th International Conference on Information Technology and Digital Application (ICITDA)*, 2025, pp. 1–8.

Appendix A. Exemplary appendix

Appendix content.

Authors and affiliations

Argo Wibowo
e-mail: argowibowo528951@mail.ugm.ac.id, argo@staff.ukdw.ac.id
ORCID: https://orcid.org/0009-0000-2689-5461
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia, Department of Information System, Universitas Kristen Duta Wacana, Indonesia

Adhistya Erna Permanasari
e-mail: adhistya@ugm.ac.id
ORCID: https://orcid.org/0000-0002-2663-4074
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia

Teguh Bharata Adji
e-mail: adji@ugm.ac.id
ORCID: https://orcid.org/0000-0001-7856-1498
Department of Electrical and Information Engineering, Universitas Gadjah Mada, Indonesia